

TCL

(트랜잭션)

- **TCL (Transcation Controll Language)**

트랜잭션을 제어하는 언어

- **트랜잭션(Transaction)**

데이터베이스에서 실행되는 논리적 작업 단위

SELECT, INSERT, UPDATE, DELETE와 같은 연산을 수행하여 데이터베이스의 상태를 변화시키는 작업의 단위로서 이러한 일련의 작업들이 발생하는 단위를 말함

• 트랜잭션의 특징(ACID)

1. Atomicity(원자성)

- 트랜잭션이 모두 반영 되든지, 모두 반영되지 않든지 해야 함
- 트랜잭션의 모든 작업은 반드시 성공 해야하며, 하나라도 에러가 발생할 경우 모든 작업에 대한 트랜잭션이 모두 취소되어야 함

2. Consistency(일관성)

- 작업의 처리 결과가 항상 일관성이 있어야 함
- 모든 무결성 제약 조건을 만족해야 함

3. Isolation(독립성)

- 두 개 이상의 트랜잭션이 실행중일 경우 다른 트랜잭션이 실행중인 트랜잭션에 끼어들 수 없음

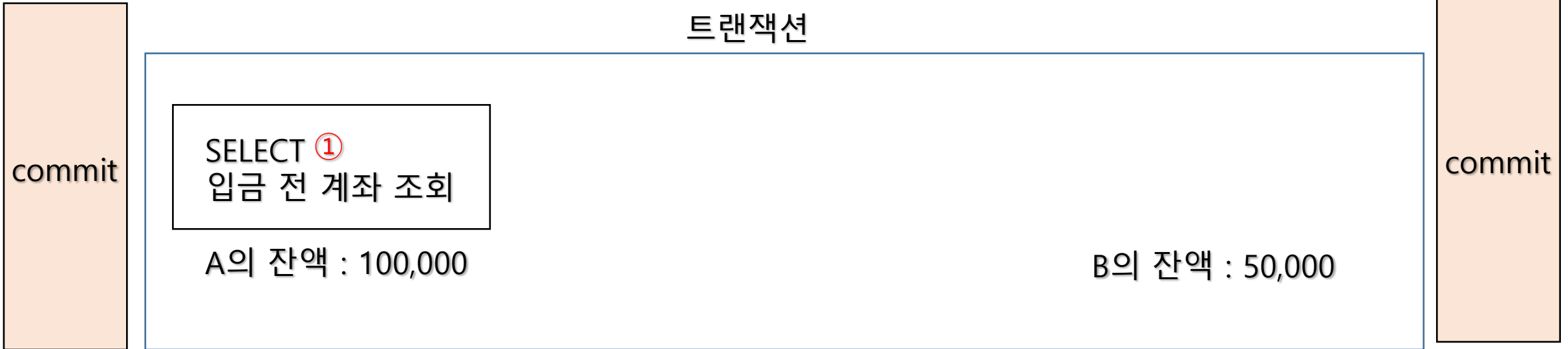
4. Durability(지속성)

- 트랜잭션이 성공했다면 결과는 영구적으로 반영되어야 함

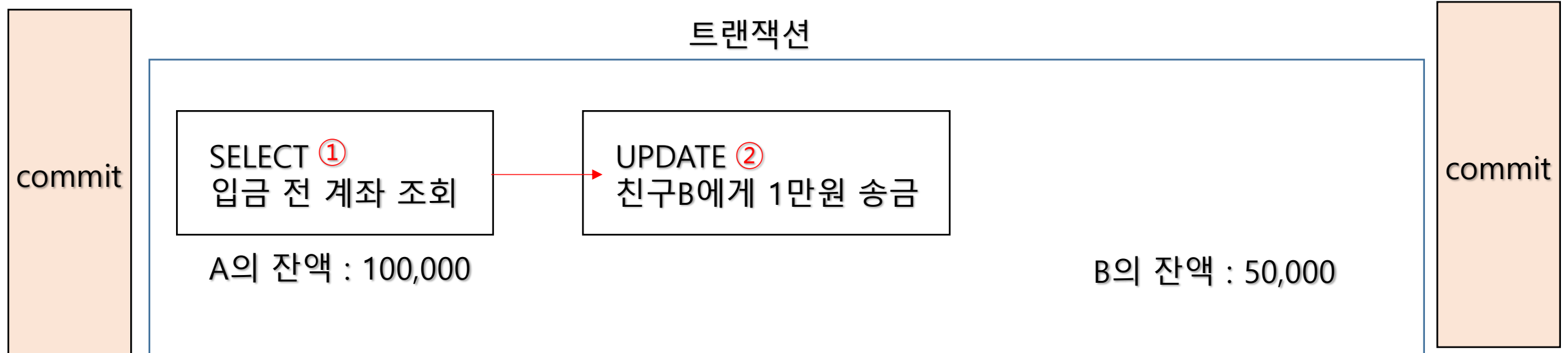
• 무결성 제약

무결성 제약	설명
개체 무결성 (Entity Integrity)	각 테이블의 기본 키가 고유(UNIQUE)하고 NULL이 아닌 값(NOT NULL)을 가지도록 보장함 * 레코드를 고유하게 식별하기 위해 필요함
참조 무결성 (Referential Integrity)	외래 키를 사용하여 테이블 간의 데이터가 일관성을 유지하도록 함 * 외래키로 들어온 참조 값은 참조중인 테이블에 반드시 존재해야 함
도메인 무결성 (Domain Integrity)	정의된 데이터 타입, 길이, 형식 등의 제약을 따르도록 보장함 * ex. 생년월일은 올바른 날짜 형식만 저장 가능
고유성 제약 (Unique Constraint)	고유한 값을 가지도록 하여 중복을 방지함 * ex. 회원이 로그인할 아이디는 고유해야 함
NULL 제약 (Null Constraint)	NULL 값을 허용하지 않도록 제한함 * 필수 데이터를 반드시 입력해야 함

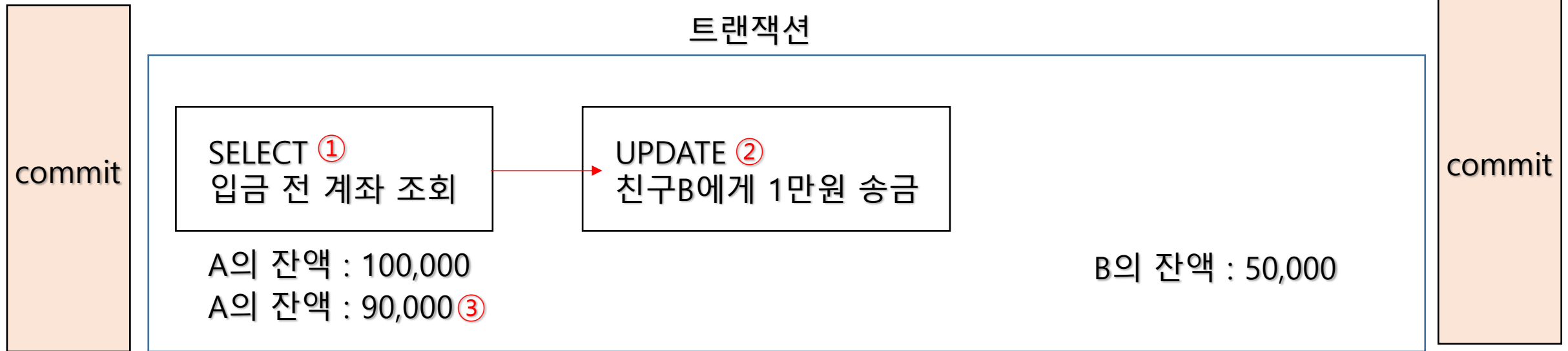
- 트랜잭션



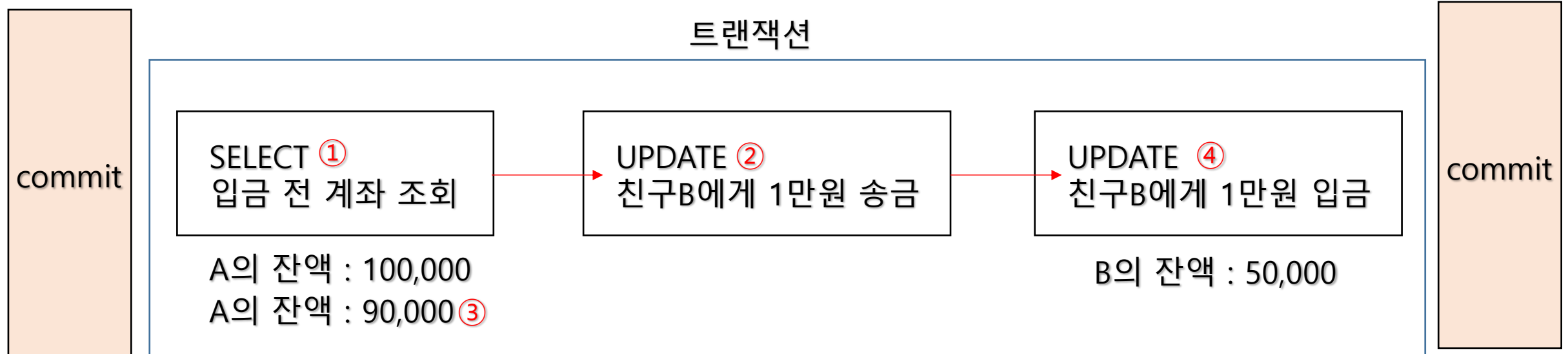
- 트랜잭션 성공 시



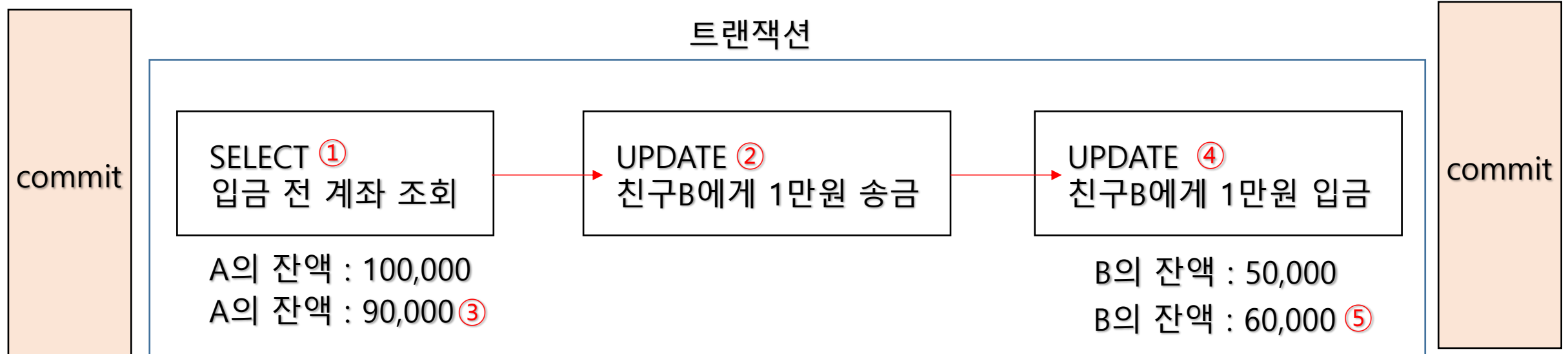
- 트랜잭션 성공 시



- 트랜잭션 성공 시

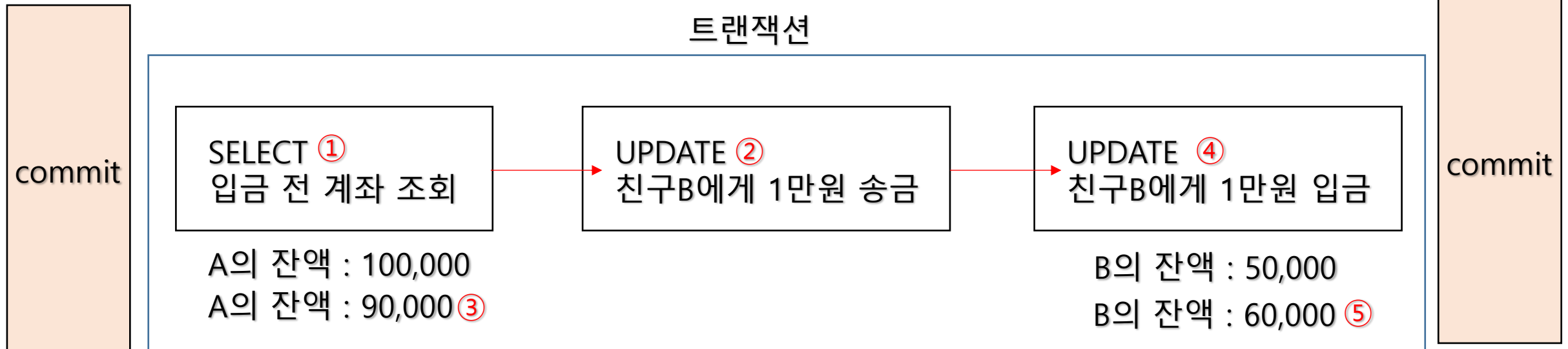


- 트랜잭션 성공 시



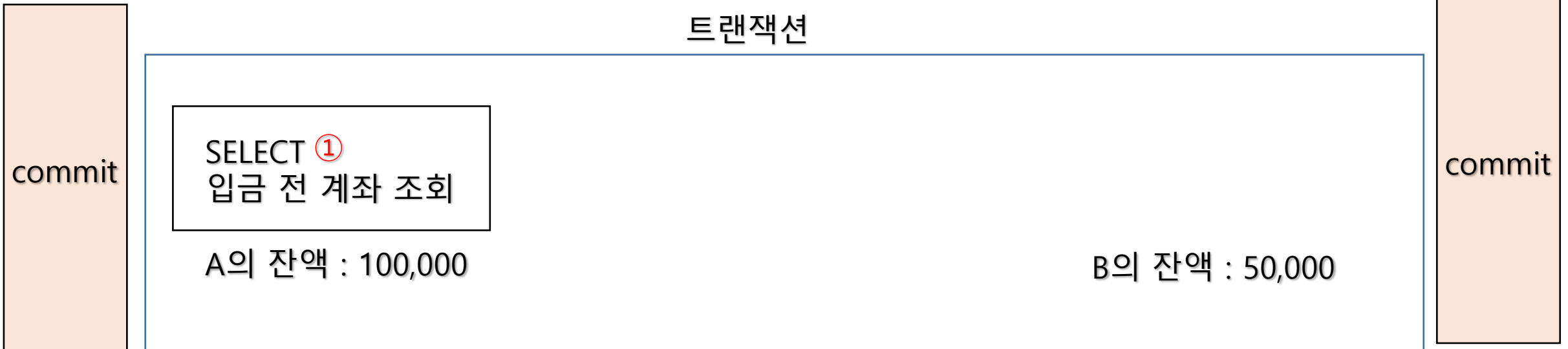
- 트랜잭션 성공 시 - COMMIT

COMMIT 실행되며
1~3 작업을 모두 저장

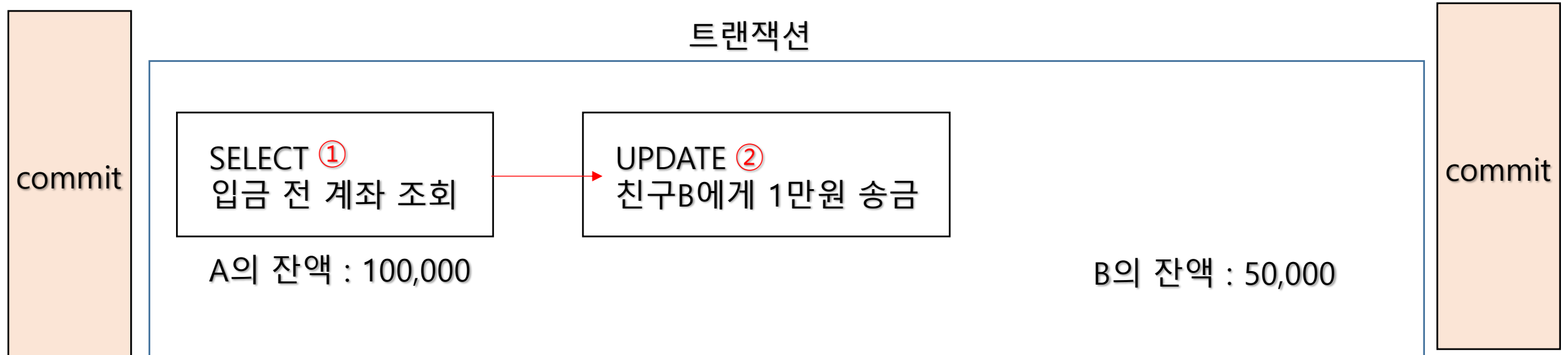


- 트랜잭션이 실패한다면?

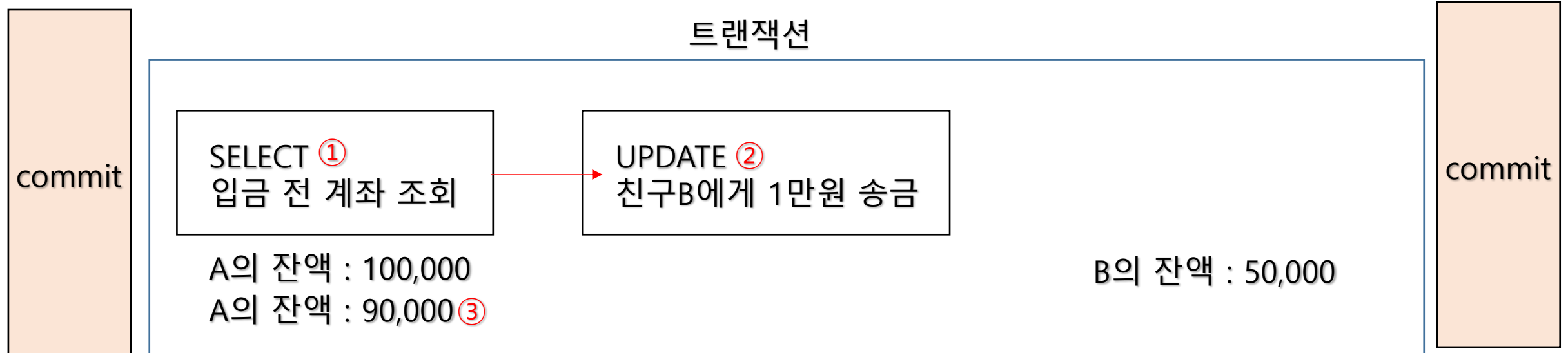
- 트랜잭션



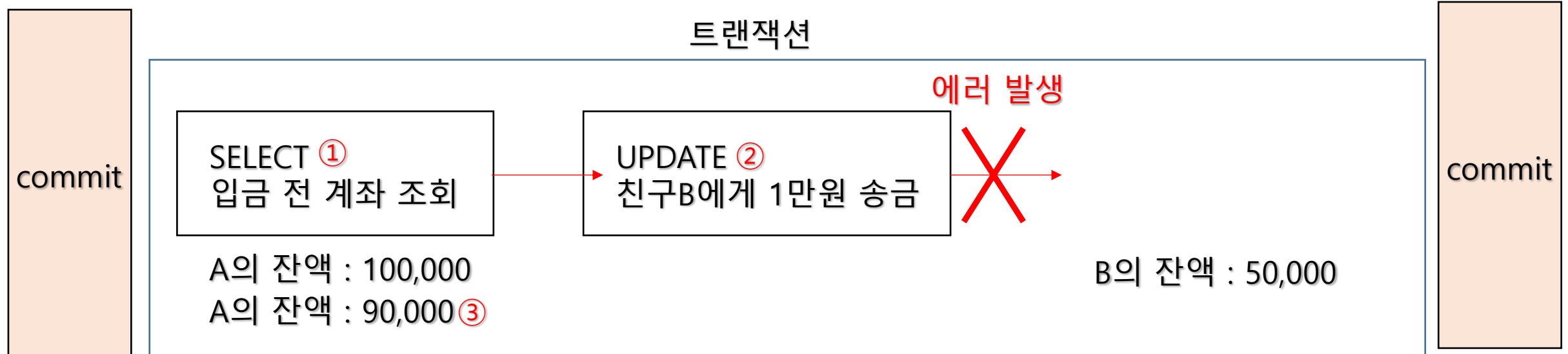
- 트랜잭션 실패 시



- 트랜잭션 실패 시



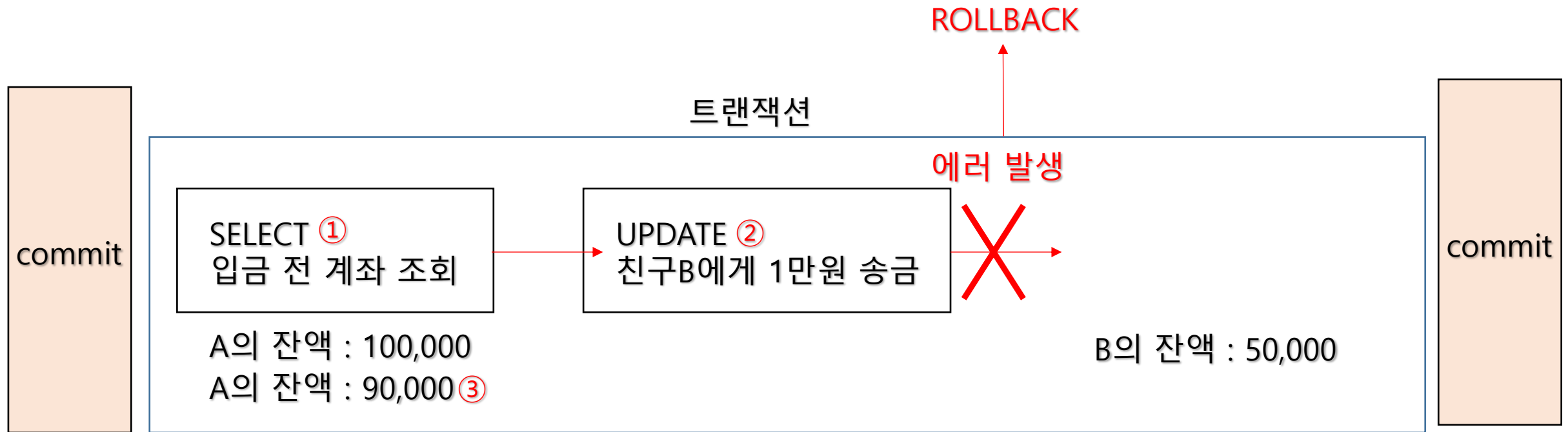
- 트랜잭션 실패 시



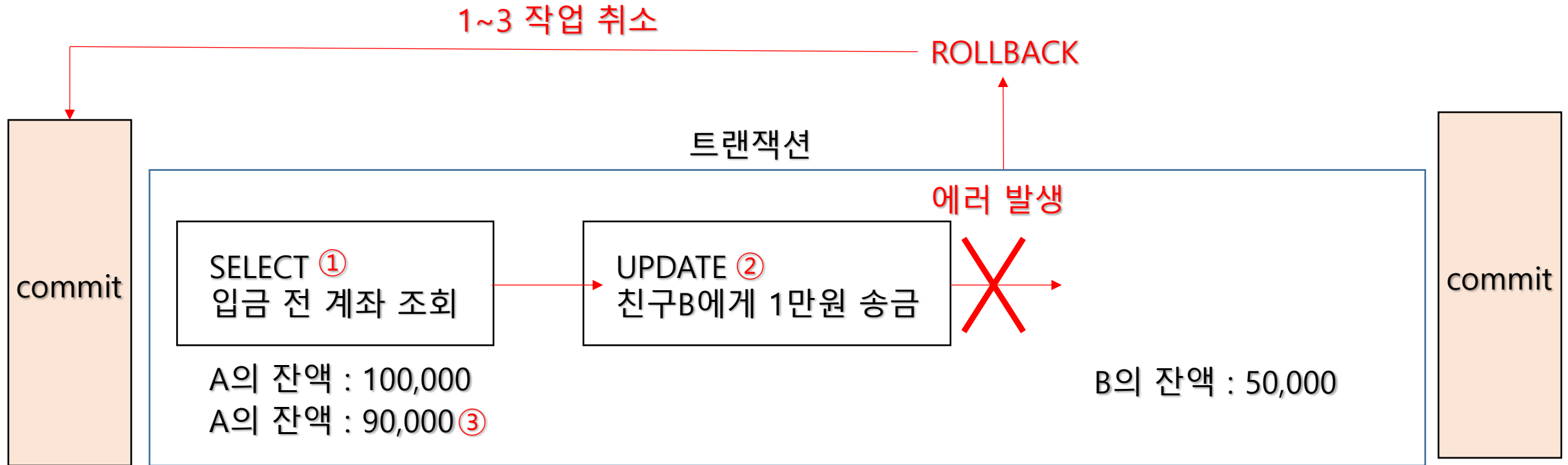
- **트랜잭션 실패 시**

A는 송금 처리되어 1만원이 차감되었으나, 송금 도중 에러가 발생하여 B는 입금을 받지 못하여 잔액이 그대로 임

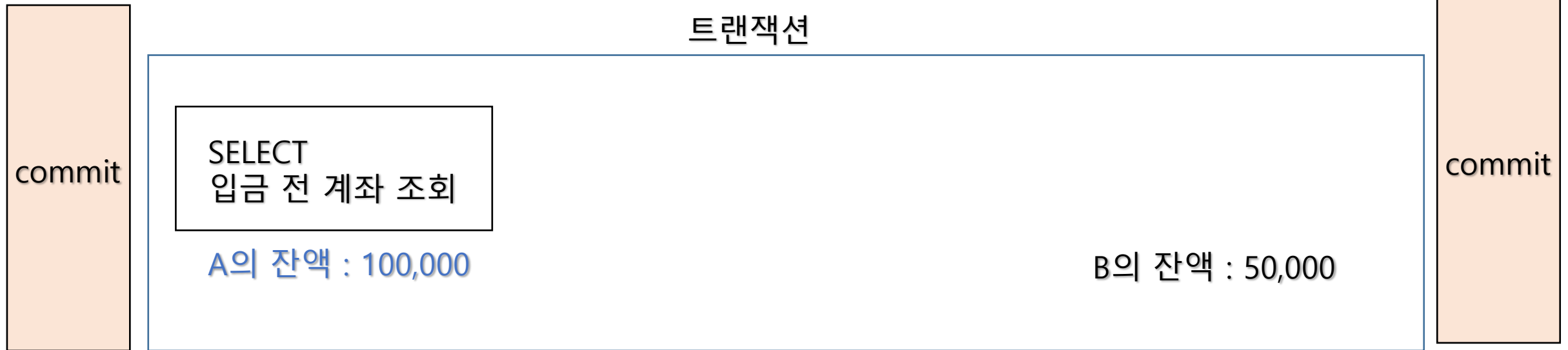
- 트랜잭션 실패 시



- 트랜잭션 실패 시



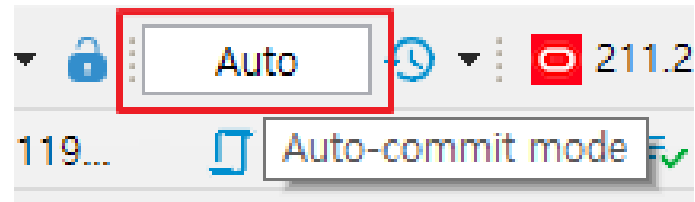
- 트랜잭션 실패 시 - ROLLBACK



DBeaver Auto-Commit

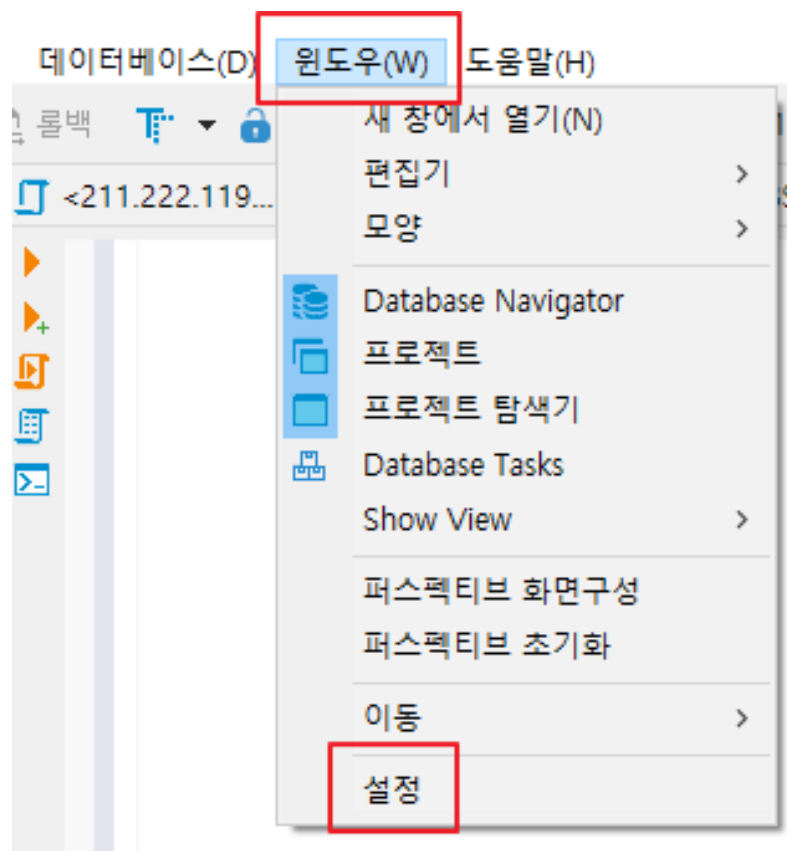
- **DBeaver Auto-Commit**

DBeaver는 기본적으로 Auto-Commit 기능이 설정되어 있어 개발자가 따로 COMMIT을 하지 않아도 매번 자동으로 COMMIT 됨



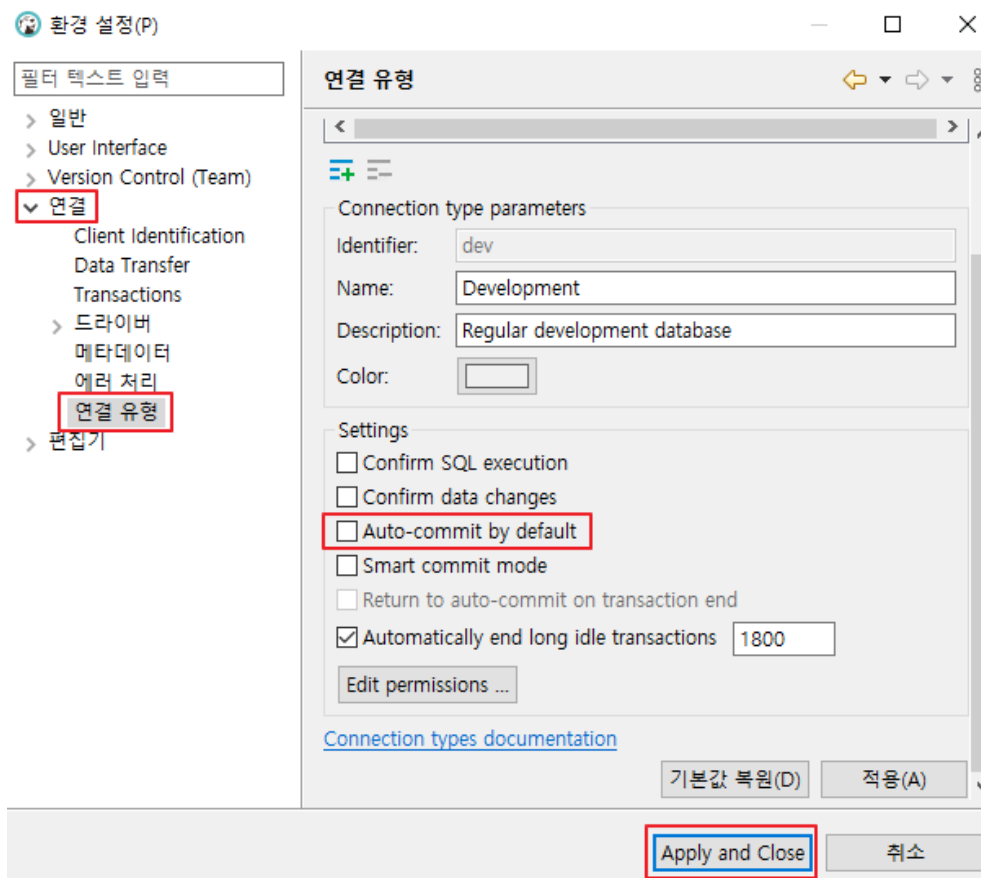
- DBeaver Auto-Commit 해제

윈도우 → 설정



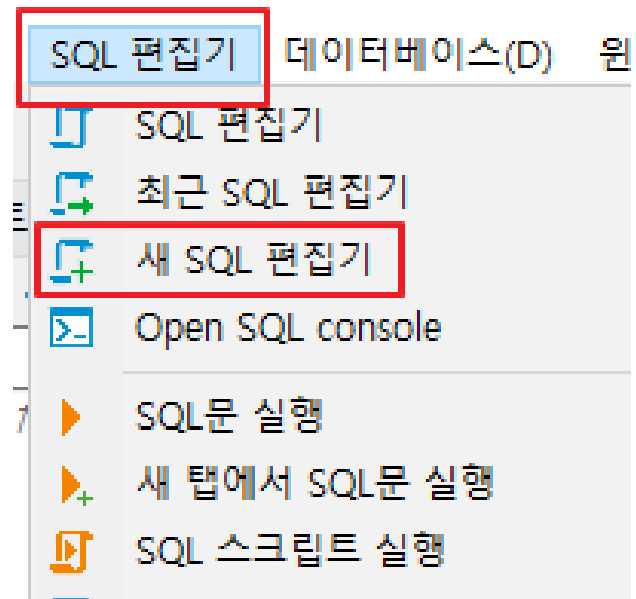
• DBeaver Auto-Commit 해제

연결 → 연결 유형 → Auto-commit by default **체크 해제** → Apply and Close



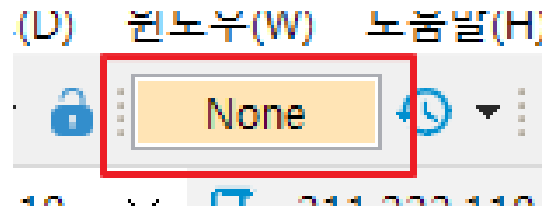
- DBeaver Auto-Commit 해제

SQL 편집기 → 새 SQL 편집기



- **DBeaver Auto-Commit 해제**

Auto → None으로 변경되어 있는 내용 확인



TCL
(COMMIT)

- **COMMIT**

모든 작업을 성공적으로 완료하여 변경된 내용을 데이터베이스에 반영하는 작업

• COMMIT 확인하기 - 1

webuser 계정으로 테이블에 데이터를 추가하고 SELECT로 삽입된 데이터 확인

* 자신이 직접 작업한 내용은 실시간으로 반영된 것처럼 보임

```
INSERT INTO WEBUSER_TB VALUES('Test1');  
INSERT INTO WEBUSER_TB VALUES('Test2');  
INSERT INTO WEBUSER_TB VALUES('Test3');  
SELECT * FROM WEBUSER_TB;
```

Results 1 X

SELECT * FROM WEBUSER_TB | Enter a SQL expression to filter results (use Ctrl+Space)

	ABC TB_ID ▼
1	Test1
2	Test2
3	Test3

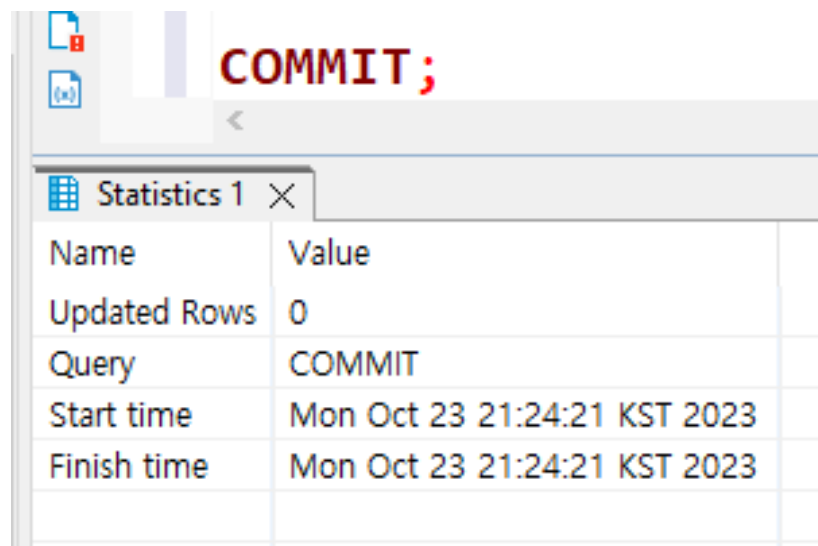
- COMMIT 확인하기 - 2

CMD창 열기 → sqlplus → webadmin 계정 접속 → SELECT * FROM webuser.webuser_tb;
입력하여 DBeaver에서 작업한 내용이 데이터베이스에 반영되지 않았음을 확인

```
SQL> select * from webuser.webuser_tb;  
선택된 레코드가 없습니다.
```

- COMMIT 확인하기 - 3

DBeaver로 되돌아와 COMMIT; 수행



The screenshot shows the DBeaver SQL editor with a red **COMMIT;** statement entered. Below the editor, the 'Statistics 1' window is open, displaying the following data:

Name	Value
Updated Rows	0
Query	COMMIT
Start time	Mon Oct 23 21:24:21 KST 2023
Finish time	Mon Oct 23 21:24:21 KST 2023

- COMMIT 확인하기 - 4

CMD로 다시 되돌아와서 다시 webuser.webuser_tb 조회

```
SQL> select * from webuser.webuser_tb;  
TB_ID  
-----  
Test1  
Test2  
Test3
```

TCL
(ROLLBACK)

- **ROLLBACK**

작업 진행 도중 에러 또는 문제가 발생하여 현재 작업 내용을 데이터베이스에 반영하지 않고, 마지막 COMMIT 시점으로 되돌림

• ROLLBACK

작업 도중 실수로 DELTE를 잘못 사용하여 전체 데이터가 삭제된 상황이라면

* 자신이 작업한 내용은 모두 삭제된것처럼 보임

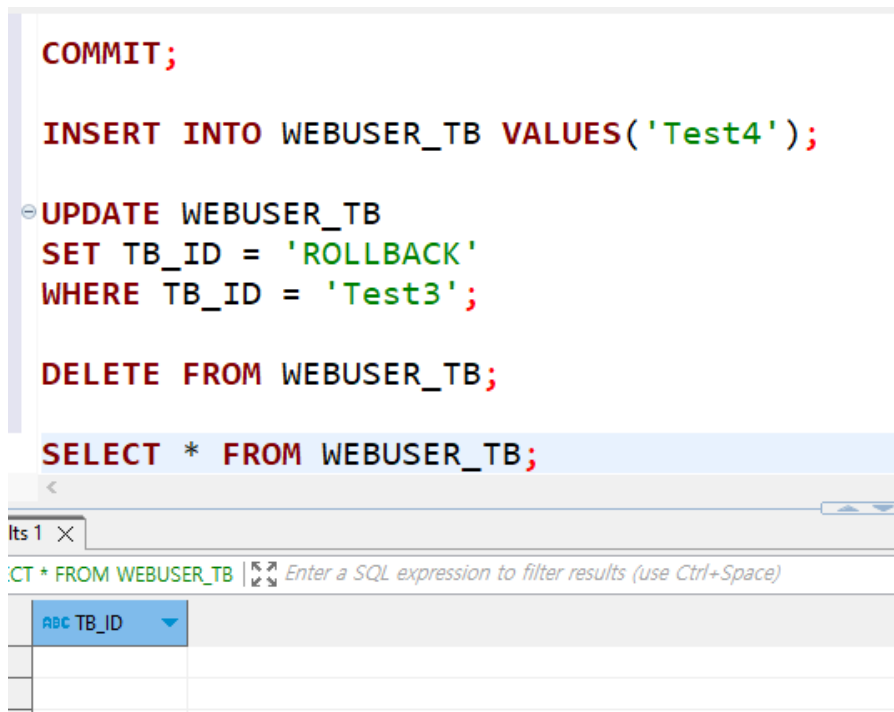
```
COMMIT;

INSERT INTO WEBUSER_TB VALUES('Test4');

UPDATE WEBUSER_TB
SET TB_ID = 'ROLLBACK'
WHERE TB_ID = 'Test3';

DELETE FROM WEBUSER_TB;

SELECT * FROM WEBUSER_TB;
```



ABC TB_ID

- ROLLBACK

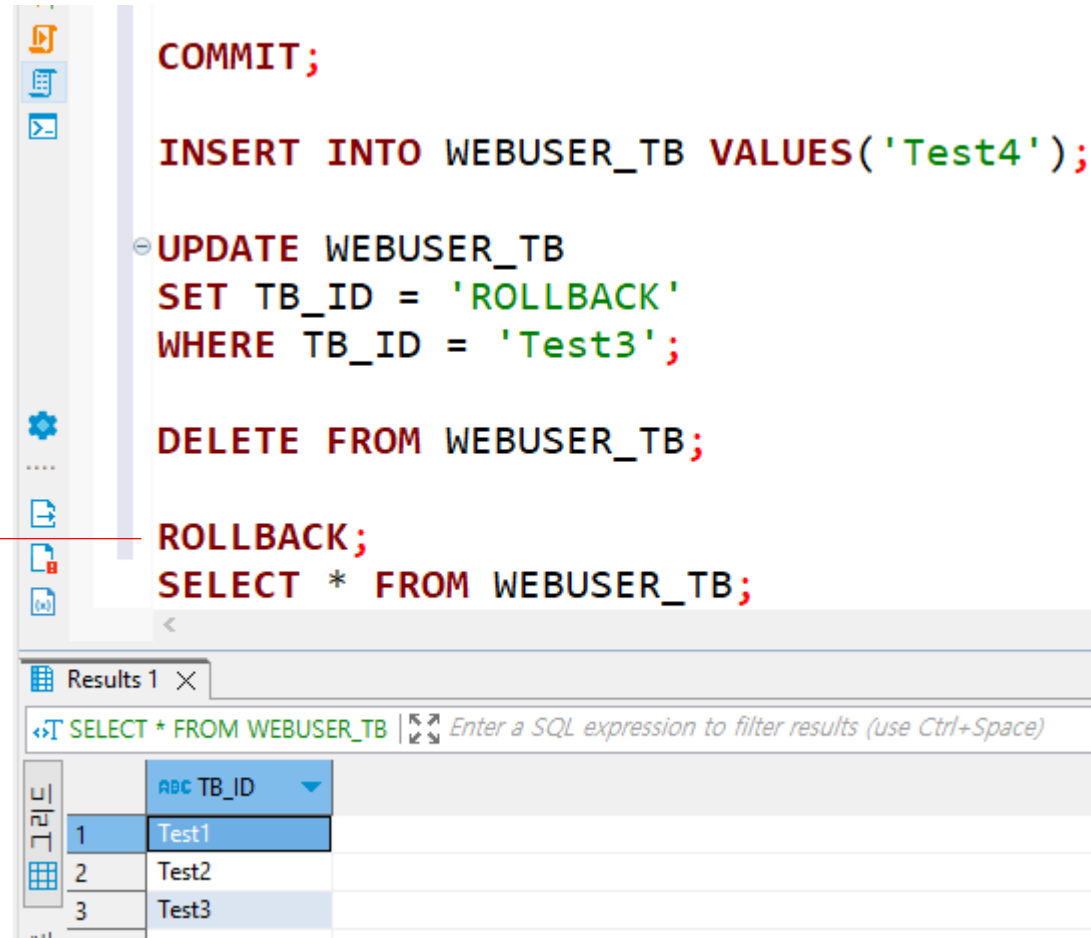
CMD에 접속되어 있는 webadmin 계정으로 확인하면 아직 데이터가 잘 보임

* COMMIT을 사용하여 데이터베이스에 반영하지 않았기 때문에

```
SQL> select * from webuser.webuser_tb;  
TB_ID  
-----  
Test1  
Test2  
Test3
```

- ROLLBACK

ROLLBACK 사용하여 마지막
COMMIT 시점으로 되돌림



```
COMMIT;  
  
INSERT INTO WEBUSER_TB VALUES('Test4');  
  
UPDATE WEBUSER_TB  
SET TB_ID = 'ROLLBACK'  
WHERE TB_ID = 'Test3';  
  
DELETE FROM WEBUSER_TB;  
  
ROLLBACK;  
SELECT * FROM WEBUSER_TB;
```

The screenshot shows a SQL IDE interface. The top pane contains a list of SQL commands. The bottom pane, titled 'Results 1', shows the output of the last command, 'SELECT * FROM WEBUSER_TB;'. The output is a table with 3 rows and 1 column, 'TB_ID'. The rows contain 'Test1', 'Test2', and 'Test3'. The 'ROLLBACK;' command is highlighted in the command list, and a red arrow points from the Korean text on the left to it.

ABC	TB_ID
1	Test1
2	Test2
3	Test3

- ROLLBACK

ROLLBACK 사용하여 마지막
COMMIT 시점으로 되돌림

The screenshot shows a SQL IDE with a list of commands on the left and a results pane at the bottom. The commands are:

```
COMMIT;  
  
INSERT INTO WEBUSER_TB VALUES('Test4');  
  
UPDATE WEBUSER_TB  
SET TB_ID = 'ROLLBACK'  
WHERE TB_ID = 'Test3';  
  
DELETE FROM WEBUSER_TB;  
  
ROLLBACK;  
SELECT * FROM WEBUSER_TB;
```

The results pane shows the output of the final SELECT statement:

ABC	TB_ID
1	Test1
2	Test2
3	Test3

TCL
(SAVEPOINT)

- **SAVEPOINT**

세이브 포인트를 만들어 롤백 시 지정한 세이브 포인트로 트랜잭션을 롤백

표현법

→ 저장 시 : SAVEPOINT 포인트명;

→ 사용 시 : ROLLBACK TO 포인트명;

- SAVEPOINT

INSERT 후 SP1 이름의 세이브 포인트를 만들어 UPDATE 진행

```
INSERT INTO WEBUSER_TB VALUES('Test4');  
SAVEPOINT SP1;
```

```
UPDATE WEBUSER_TB  
SET TB_ID = 'ROLLBACK'  
WHERE TB_ID = 'Test3';
```

```
SELECT * FROM WEBUSER_TB;
```

Results 1 X

SELECT * FROM WEBUSER_TB | Enter a SQL expression to filter results (use Ctrl+Space)

	ABC TB_ID ▼
1	Test1
2	Test2
3	ROLLBACK
4	Test4

- SAVEPOINT

ROLLBACK 사용하여
SP1 지점으로 되돌림

```
INSERT INTO WEBUSER_TB VALUES('Test4');  
SAVEPOINT SP1;  
  
UPDATE WEBUSER_TB  
SET TB_ID = 'ROLLBACK'  
WHERE TB_ID = 'Test3';  
  
SELECT * FROM WEBUSER_TB;  
ROLLBACK TO SP1;  
SELECT * FROM WEBUSER_TB;
```

Results 1 X

'SELECT * FROM WEBUSER_TB' Enter a SQL expression to filter results (use Ctrl+Space)

	ABC TB_ID
1	Test1
2	Test2
3	Test3
4	Test4

- SAVEPOINT

ROLLBACK 사용하여
SP1 지점으로 되돌림

```
INSERT INTO WEBUSER_TB VALUES('Test4');
SAVEPOINT SP1;

UPDATE WEBUSER_TB
SET TB_ID = 'ROLLBACK'
WHERE TB_ID = 'Test3';

SELECT * FROM WEBUSER_TB;
ROLLBACK TO SP1;
SELECT * FROM WEBUSER_TB;
```

Results 1 X

SELECT * FROM WEBUSER_TB | Enter a SQL expression to filter results (use Ctrl+Space)

	ABC TB_ID
1	Test1
2	Test2
3	Test3
4	Test4

- 참고

한번 사용 후 SP1 지점으로 다시 되돌릴 수 없음.
만약 다시 되돌리고 싶다면 SAVEPOINT SP1을 한번 더 선언 해주어야 함

- SAVEPOINT

SP1 지점으로 롤백 후 다시 ROLLBACK; 을 입력하게 되면 세이브포인트 지점을 지정하지 않았기 때문에 마지막 COMMIT 지점으로 롤백됨

```
COMMIT;  
  
INSERT INTO WEBUSER_TB VALUES('Test4');  
SAVEPOINT SP1;  
  
UPDATE WEBUSER_TB  
SET TB_ID = 'ROLLBACK'  
WHERE TB_ID = 'Test3';  
  
SELECT * FROM WEBUSER_TB;  
ROLLBACK TO SP1;  
SELECT * FROM WEBUSER_TB;  
ROLLBACK;  
SELECT * FROM WEBUSER_TB;
```

Results 1 ×

*SELECT * FROM WEBUSER_TB | Enter a SQL expression to filter results (use Ctrl+Space)

	ABC TB_ID
1	Test1
2	Test2
3	Test3

TCL
(실습)

- Q1. 실습 테이블

```
CREATE TABLE accounts (  
    account_id NUMBER PRIMARY KEY,  
    account_name VARCHAR(255),  
    balance NUMBER  
);
```

- Q1. 실습 데이터 삽입

```
INSERT INTO ACCOUNTS VALUES (1, 'Alice', 10000);  
INSERT INTO ACCOUNTS VALUES (2, 'Bob', 15000);  
INSERT INTO ACCOUNTS VALUES (3, 'Charlie', 20000);
```

• Q1. 실습

1. Alice의 계좌에서 Bob이 계좌로 5000원을 이체합니다.
 - UPDATE문을 이용하여 Alice의 blance 컬럼의 값을 5000 뺍니다.
 - ex) SET balance = balance-5000
 - UPDATE문을 이용하여 Bob의 balance 컬럼의 값을 5000 더합니다.
2. SAVEPOINT를 작성하세요.
3. Charlie의 계좌에서 10000원을 출금하고, 그 후 ROLLBACK을 사용하여 해당 출금을 취소합니다.
 - 세이브 포인트로 롤백하세요.
4. 이후 COMMIT을 사용하여 데이터베이스에 반영하세요.